

Sistema gestión fintech POO

Diseñar e implementar un sistema de gestión para una fintech que ofrece servicios bancarios a sus clientes. El sistema debe permitir la creación de cuentas para los clientes nuevos, realizar movimientos como depósitos y retiros, y mantener un registro de los movimientos realizados en cada cuenta. Gestión de clientes: El sistema debe permitir la creación, modificación y eliminación de clientes. Cada cliente debe tener un identificador único, nombre, apellido y DNI. El usuario debe ingresar al sistema con su correo electrónico

Te dejo una **primera versión formal** de la descripción del proyecto:



Proyecto: Sistema de Gestión para Fintech

Descripción general

El proyecto consiste en diseñar e implementar un sistema web de gestión para una fintech que ofrece servicios bancarios básicos a sus clientes. El sistema permitirá administrar clientes, crear cuentas bancarias, realizar movimientos como depósitos y retiros, consultar saldos y mantener un historial de operaciones realizadas.

Esta primera versión será desarrollada utilizando **HTML, CSS y JavaScript**, aplicando los principios de la **Programación Orientada a Objetos (POO)**. El objetivo principal es construir una aplicación sencilla, clara y funcional, que permita representar entidades reales del sistema financiero mediante clases, objetos, métodos y relaciones entre componentes.

Objetivos del sistema

El sistema deberá permitir:

- Registrar nuevos clientes.
- Modificar los datos de clientes existentes.

- Eliminar clientes del sistema.
- Crear cuentas asociadas a clientes.
- Consultar el saldo actual de una cuenta.
- Realizar depósitos.
- Realizar retiros.
- Registrar cada movimiento realizado.
- Visualizar el historial de movimientos de una cuenta.
- Permitir el ingreso al sistema mediante correo electrónico y password.

Gestión de clientes

Cada cliente será representado como un objeto dentro del sistema. Los datos principales de cada cliente serán:

- Identificador único.
- Nombre.
- Apellido.
- DNI.
- Correo electrónico.
- Password.

El sistema deberá validar que cada cliente tenga un identificador único y que pueda ser buscado, modificado o eliminado desde la interfaz web.

Gestión de cuentas

Cada cliente podrá tener una o más cuentas asociadas. Cada cuenta deberá tener:

- Código único de cuenta.
- Saldo inicial.
- Cliente asociado.
- Listado de movimientos realizados.

La cuenta será responsable de administrar su saldo y de registrar cada operación realizada sobre ella.

Movimientos bancarios

El sistema permitirá realizar dos tipos principales de movimientos:

- Depósito.
- Retiro.

Cada movimiento deberá registrar:

- Tipo de transacción.
- Monto.
- Fecha de realización.

Cuando se realice un depósito, el saldo de la cuenta aumentará.

Cuando se realice un retiro, el saldo disminuirá, siempre que exista saldo suficiente.

Consulta de saldo

Los clientes podrán consultar el saldo actual de sus cuentas. Esta información deberá actualizarse automáticamente luego de cada depósito o retiro realizado.

Enfoque orientado a objetos

El proyecto se desarrollará aplicando Programación Orientada a Objetos. Para ello, se podrán definir clases como:

- Cliente.
- Cuenta.
- Movimiento.
- SistemaFintech.

Cada clase tendrá atributos y métodos propios, respetando buenas prácticas como modularización, encapsulamiento y separación de responsabilidades.

Interfaz de usuario

La aplicación contará con una interfaz web amigable, desarrollada con HTML y CSS. Desde la interfaz, el usuario podrá:

- Iniciar sesión.
- Registrar clientes.
- Listar clientes.
- Crear cuentas.
- Realizar depósitos y retiros.

- Consultar saldos.
- Ver movimientos realizados.

Buenas prácticas a aplicar

Durante el desarrollo se deberán aplicar buenas prácticas de programación, tales como:

- Código modular y organizado.
- Uso de clases y objetos.
- Métodos con responsabilidades claras.
- Comentarios descriptivos.
- Validación básica de datos.
- Separación entre estructura HTML, estilos CSS y lógica JavaScript.

Alcance inicial del proyecto

En esta primera versión, el sistema funcionará del lado del navegador, utilizando JavaScript para manejar los datos en memoria. No se utilizará todavía una base de datos ni backend. El objetivo inicial será comprender la lógica del sistema y aplicar correctamente los conceptos de POO.

En futuras versiones, el proyecto podrá ampliarse incorporando almacenamiento persistente, conexión con una base de datos, autenticación real de usuarios y un backend desarrollado con tecnologías como Java, Node.js o Spring Boot.

Podés usar esta descripción como **consigna inicial del trabajo práctico** o como introducción del proyecto para los alumnos.



dame un primer ejemplo de las vistas del proyecto y el modelo de clases sugerido

Claro. Una primera estructura podría ser esta:

Vistas sugeridas del proyecto

1. Vista de Login

Permite que el usuario ingrese con correo y password.

HTML

```
<section id="vista-login">
  <h2>Ingreso al sistema</h2>

  <input type="email" id="email" placeholder="Correo electrónico">
  <input type="password" id="password" placeholder="Password">

  <button>Ingresar</button>
</section>
```

2. Vista principal / Dashboard

Pantalla inicial luego del login.

HTML

```
<section id="vista-dashboard">
  <h2>Panel principal</h2>

  <button>Gestionar clientes</button>
  <button>Gestionar cuentas</button>
  <button>Realizar movimiento</button>
  <button>Consultar saldo</button>
</section>
```

3. Vista de gestión de clientes

Permite crear, modificar, eliminar y listar clientes.

HTML

```
<section id="vista-clientes">
  <h2>Gestión de clientes</h2>

  <input type="text" id="nombre" placeholder="Nombre">
  <input type="text" id="apellido" placeholder="Apellido">
  <input type="text" id="dni" placeholder="DNI">
  <input type="email" id="correo" placeholder="Correo electrónico">
  <input type="password" id="clave" placeholder="Password">
```

```
<button>Agregar cliente</button>
<button>Modificar cliente</button>
<button>Eliminar cliente</button>

<h3>Listado de clientes</h3>
<div id="lista-clientes"></div>
</section>
```

4. Vista de gestión de cuentas

Permite crear cuentas asociadas a clientes.

HTML

```
<section id="vista-cuentas">
  <h2>Gestión de cuentas</h2>

  <input type="text" id="codigoCuenta" placeholder="Código de cuenta">
  <input type="number" id="saldoInicial" placeholder="Saldo inicial">
  <input type="text" id="dniCliente" placeholder="DNI del cliente">

  <button>Crear cuenta</button>

  <h3>Cuentas registradas</h3>
  <div id="lista-cuentas"></div>
</section>
```

5. Vista de movimientos

Permite realizar depósitos y retiros.

HTML

```
<section id="vista-movimientos">
  <h2>Movimientos de cuenta</h2>

  <input type="text" id="codigoCuentaMovimiento" placeholder="Código de cuenta">
  <input type="number" id="monto" placeholder="Monto">

  <button>Depositar</button>
  <button>Retirar</button>

  <h3>Historial de movimientos</h3>
  <div id="historial-movimientos"></div>
</section>
```

6. Vista de consulta de saldo

Permite consultar el saldo actual de una cuenta.

HTML

```
<section id="vista-saldo">
  <h2>Consulta de saldo</h2>

  <input type="text" id="codigoCuentaSaldo" placeholder="Código de cuenta">
  <button>Consultar saldo</button>

  <p id="resultado-saldo"></p>
</section>
```

Modelo de clases sugerido

Clase Cliente

Representa a una persona registrada en la fintech.

JavaScript

```
class Cliente {
  constructor(id, nombre, apellido, dni, email, password) {
    this.id = id;
    this.nombre = nombre;
    this.apellido = apellido;
    this.dni = dni;
    this.email = email;
    this.password = password;
  }
}
```

Clase Cuenta

Representa una cuenta bancaria asociada a un cliente.

JavaScript

```
class Cuenta {
  constructor(codigo, saldoInicial, cliente) {
    this.codigo = codigo;
    this.saldo = saldoInicial;
    this.cliente = cliente;
    this.movimientos = [];
  }
}
```

```
depositar(monto) {  
    this.saldo += monto;  
  
    const movimiento = new Movimiento("Depósito", monto, new Date());  
    this.movimientos.push(movimiento);  
}  
  
retirar(monto) {  
    if (monto <= this.saldo) {  
        this.saldo -= monto;  
  
        const movimiento = new Movimiento("Retiro", monto, new Date());  
        this.movimientos.push(movimiento);  
    } else {  
        alert("Saldo insuficiente");  
    }  
}  
  
consultarSaldo() {  
    return this.saldo;  
}  
}
```

Clase Movimiento

Representa una operación realizada sobre una cuenta.

📄 JavaScript

```
class Movimiento {  
    constructor(tipo, monto, fecha) {  
        this.tipo = tipo;  
        this.monto = monto;  
        this.fecha = fecha;  
    }  
}
```

Clase SistemaFintech

Administra clientes y cuentas.

📄 JavaScript

```
class SistemaFintech {  
    constructor() {  
        this.clientes = [];  
    }  
}
```



```
    this.cuentas = [];  
  }  
  
  agregarCliente(cliente) {  
    this.clientes.push(cliente);  
  }  
  
  eliminarCliente(dni) {  
    this.clientes = this.clientes.filter(cliente => cliente.dni !== dni);  
  }  
  
  buscarClientePorDni(dni) {  
    return this.clientes.find(cliente => cliente.dni === dni);  
  }  
  
  agregarCuenta(cuenta) {  
    this.cuentas.push(cuenta);  
  }  
  
  buscarCuentaPorCodigo(codigo) {  
    return this.cuentas.find(cuenta => cuenta.codigo === codigo);  
  }  
  
  login(email, password) {  
    return this.clientes.find(  
      cliente => cliente.email === email && cliente.password === password  
    );  
  }  
}
```

La relación principal sería:

```
Cliente 1 ----- * Cuenta  
Cuenta 1 ----- * Movimiento  
SistemaFintech contiene listas de Clientes y Cuentas
```

